1. Rendering single line of text.

Images and other embedded objects sholud respect BiDi reordering too.
Replace them with "Object Replacement Character".

Input

Unicode string: (UTF-16)

T	e	s	t	␣	Ξ	ε	σ	κ	ε	π	ά	ζ	ω	␣	ر	ح	ي	ر	ح	ي	م	␣	e	[obj]	t	e	́	n	d	↵	i	t	ş	ţ	ş	ţ
0054	0065	0073	0074	0020	039E	03B5	03C3	03BA	03B5	03C0	03AC	03B6	03C9	0020	0631	062D	064A	0645	0020	0631	062D	064A	0645	0020	0065	FFFC	0074	0065	0301	006E	0064	000A	0069	0074	015F	0162	015F	0162

Base direction:

TEXT_DIRECTION_AUTO
Text direction is detected based on first encountered code point (AUTO), current locale (LOCALE) or specified by user. Base direction is used to initialize BiDi iterator .

OpenType font features: (optional*)

OT features tags: dlig | {none}

Font: (optional*)

Sans | Nastaliq | Sans

Language: (optional*)

Language tags: {auto} | en | ro

* in case of plain (non attributed) string font/language and features are specified for entire string.

1.1. Split the string into BiDi and Script runs, then combine attribute/bidi and script runs into monotone runs.

BiDi Runs: LTR, 0 | RTL, 1 | LTR, 2 — BiDi

Script Runs: Script tags: Latn | Grek | Arab | Latn — Script Iterator

1.1.1. Combine into monotone runs.

Monotone Runs:	0	1	3	2	4	5	6	7
Order:	0	1	3	2	4	5	6	7
Script:	Latn	Grek	Arab	Arab	Arab	Latn	Latn	Latn
Direction:	LTR	LTR	RTL	RTL	LTR	LTR	LTR	LTR
Language:	{auto}	{auto}	{auto}	{auto}	{auto}	{auto}	en	ro
Font:	Sans	Sans	Sans	Nastaliq	Nastaliq	Sans	Sans	Sans
Font features:	dlig	{none}	{none}	{none}	{none}	{none}	{none}	{none}

1.2. Shape runs in the order provided by BiDi algorithm.

HarfBuzz

Glyph

Ligature forced by "dlig" OpenType feature tag

Input string code units

Grapheme cluster

T	e	st	␣
37	48	356	3
0...0	1...1	2...3	4...4

Shaping failed (some returned glyphs have index "0").

1.2.1. Shape runs with fallback font.

Ξ	ε	σ	κ	π	ε	ά	ζ	ω	␣
0	0	0	0	π 71	0	0	0	0	3
5...5	6...6	7...7	8...8	9...9	10...10	11...11	12...12	13...13	14...14
16b	181	18f	186	181	18c	178	182	195	3
5...5	6...6	7...7	8...8	9...9	10...10	11...11	12...12	13...13	14...14

HarfBuzz

́	́	́	́	́
118	13	183	3d7	1eb	ec
23...23	22...22		21...21		20...20

́	́	́	́	́
3	1ed	1fa	1c0	31
19...19	18...18	17...17	16...16	15...15

́
3
24...24

Multiple code points combined into one cluster

No cluster for control code point (\n)

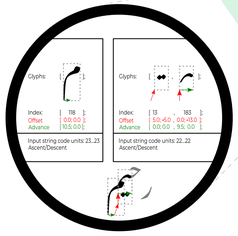
e	ø	t	é	n	d	i	t
48		57	ab	51	47	4c	57
25...25	26...26	27...27	28...29	30...30	31...31	33...33	34...34

Replace with actual image

ş	ţ
64	67
35...35	36...36

Same code point can be shaped into different glyphs!

ş	ţ
74	87
37...37	38...38

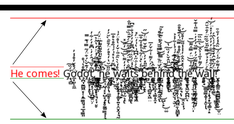


HarfBuzz returns font specific glyph indicies, offsets, advanced and character ranges that corrisponds to the cluster.
Ascent and descent of cluster is calculated based on the font ascent/descent and glyph offsets.

"HarfBuzz is a text-shaping engine. If you give HarfBuzz a font and a string containing a sequence of Unicode code points, HarfBuzz selects and positions the corresponding glyphs from the font, applying all of the necessary layout rules and font features. HarfBuzz then returns the string to you in the form that is correctly arranged for the language and writing system."
What is HarfBuzz?

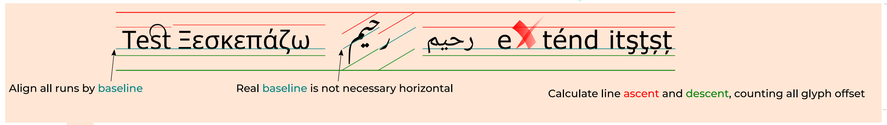
ICU BiDi API References
UAX #9: The Bidirectional Algorithm

Character - ambiguous term, can mean anything
Code unit - atomic unit of the encodeing (16-bits in case of UTF-16), represent a full code point, or part of a code point
Code point - atomic unit of the Unicode standard (independent of encoding)
Glyph - an image, atomic element of the font (font specific, doesn't have any meaning outside of the font)
Grapheme cluster - atomic unit of a writing system



In some cases difference between line and font ascent/descent can be significant.

1.3. Baseline align, ascent and descent.



T	e	st	␣	Ξ	ε	σ	κ	π	ε	ά	ζ	ω	␣	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́	́
37	48	356	3	16b	181	18f	186	181	18c	178	182	195	3	118	13	183	3d7	1eb	ec	3	1ed	1fa	1c0	31	3	48	5b	57	ab	51	47	4c	57	64	67	74	87
0...0	1...1	2...3	4...4	5...5	6...6	7...7	8...8	9...9	10...10	11...11	12...12	13...13	14...14	23...23	22...22		21...21		20...20	19...19	18...18	17...17	16...16	15...15	24...24	25...25	26...26	27...27	28...29	30...30	31...31	33...33	34...34	35...35	36...36	37...37	38...38

1.4. Draw glyphs left-to-right using advances and offsets of the glyphs.

Output

Test Ξεσκεπάζω رحيم رحيم e text itştşţ

Optional Attributes — Optional Attribute Run Iteration

BiDi iterator init. — BiDi Run Iteration

```
UBiDi *bidi_iter = ubidi_openSized(data.size, 0, &err);
if (U_FAILURE(err)) {

}
switch (base_direction) {
    case TEXT_DIRECTION_LOCALE: {
        ubidi_setPara(bidi_iter, data, data.size, uloc_isRightToLeft(locale) ? UBIDI_RTL : UBIDI_LTR, NULL, &err);
    } break;
    case TEXT_DIRECTION_LTR: {
        ubidi_setPara(bidi_iter, data, data.size, UBIDI_LTR, NULL, &err);
    } break;
    case TEXT_DIRECTION_RTL: {
        ubidi_setPara(bidi_iter, data, data.size, UBIDI_RTL, NULL, &err);
    } break;
    case TEXT_DIRECTION_AUTO: {
        UBiDiDirection direction = ubidi_getBaseDirection(data, data.size);
        if (direction != UBIDI_NEUTRAL) {
            ubidi_setPara(bidi_iter, data, data.size, direction, NULL, &err);
        } else {
            ubidi_setPara(bidi_iter, data, data.size, uloc_isRightToLeft(locale) ? UBIDI_RTL : UBIDI_LTR, NULL, &err);
        }
    } break;
}
if (U_FAILURE(err)) {

}
```

Generate BiDi runs in the "visual" (LTR) order.

```
int64_t bidi_run_count = ubidi_countRuns(bidi_iter, &err);
if (U_FAILURE(err)) {

}

for (int64_t i = 0; i < bidi_run_count; i++) {
    int32_t bidi_run_start = 0;
    int32_t bidi_run_length = 0;
    hb_direction_t bidi_run_direction = (ubidi_getVisualRun(bidi_iter, i, &bidi_run_start, &bidi_run_length) == UBIDI_LTR) ? HB_DIRECTION_LTR : HB_DIRECTION_RTL;
    bidi_run_start                                  bidi_run_start + bidi_run_length
                        bidi_run_direction, i

}
```

Script Iterator Definition
Script Iterator Implementation
Script Run Iteration

```
ScriptIterator *script_iter = new ScriptIterator(data, 0, data.size);
```

Generate script runs

```
while (script_iter->next()) {
    int32_t script_run_start = script_iter->get_start();
    int32_t script_run_end = script_iter->get_end();
    hb_script_t script_run_script = script_iter->get_script();
    script_run_start                                    script_run_end
                        script_run_script
}
```

Shape monotone run

```
hb_buffer_set_direction(hb_buffer, p_run_direction);
hb_buffer_set_flags(hb_buffer, (hb_buffer_flags_t)(HB_BUFFER_FLAG_DEFAULT | (p_run_start == 0 ? HB_BUFFER_FLAG_BOT : 0) | (p_run_end == data.size ? HB_BUFFER_FLAG_EOT : 0)));
hb_buffer_set_script(hb_buffer, p_run_script);
hb_buffer_set_language(hb_buffer, language);
hb_buffer_add_utf16(hb_buffer, (const uint16_t *)data, data.size, p_run_start, p_run_end - p_run_start);
hb_shape(hb_font, hb_buffer, font_features.empty() ? NULL : &font_features[0], font_features.size());

unsigned int glyph_count;
hb_glyph_info_t *glyph_info = hb_buffer_get_glyph_infos(hb_buffer, &glyph_count);
hb_glyph_position_t *glyph_pos = hb_buffer_get_glyph_positions(hb_buffer, &glyph_count);

if (glyph_count > 0) {
    uint32_t last_cluster_id = -1;
    for (int64_t i = 0; i < glyph_count; i++) {
        if (glyph_info[i].cluster >= data.size) {

            }
            if (last_cluster_id != glyph_info[i].cluster) {
                //Start new cluster
                    glyph_info[i]
                    glyph_pos[i]
                glyph_info[i].cluster ... glyph_info[i].cluster

                //Set previous logical cluster end limit
                if (i != 0) {
                    if (p_run_direction == HB_DIRECTION_LTR) {
                        run_clusters[run_clusters.size() - 1].end = glyph_info[i].cluster - 1;
                    } else {
                        new_cluster.end = run_clusters[run_clusters.size() - 1].start - 1;
                    }
                }

                run_clusters.push_back(new_cluster);

                last_cluster_id = glyph_info[i].cluster;
            } else {
                //Add glyphs to existing cluster
                    glyph_info[i]
                    glyph_pos[i]

            }
        }
    //Set last logical cluster end limit
    if (run_clusters.size() > 0) {
        if (p_run_direction == HB_DIRECTION_LTR) {
            run_clusters[run_clusters.size() - 1].end = p_run_end - 1;
        } else {
            run_clusters[0].end = p_run_end - 1;
        }
    }
}

//Reshape sub-runs with invalid clusters using fallback fonts
if (run_clusters.size() > 0) {
    uint32_t failed_subrun_start = p_run_end + 1;
    uint32_t failed_subrun_end = p_run_start;
    for (int64_t i = 0; i < run_clusters.size(); i++) {
        if (run_clusters[i].valid) {
            if (failed_subrun_start != p_run_end + 1) {
                Shape runs with fallback font.
                failed_subrun_start = p_run_end + 1;
                failed_subrun_end = p_run_start;
            }
            visual.push_back(run_clusters[i]);
        } else {
            if (failed_subrun_start >= run_clusters[i].start) failed_subrun_start = run_clusters[i].start;
            if (failed_subrun_end <= run_clusters[i].end) failed_subrun_end = run_clusters[i].end;
        }
    }
    if (failed_subrun_start != p_run_end + 1) {
        Shape runs with fallback font.
    }
}
```

Shaping and font fallback (Attributed String Variant)

Line ascent/descent — Baseline, ascent and dsecent calulation

```
if (visual.size() > 0) {

    for (int64_t i = 0; i < visual.size(); i++) {
        //Calc max ascent / descent
        if (max_ascent < visual[i].ascent) {
            max_ascent = visual[i].ascent;
        }
        if (max_descent < visual[i].descent) {
            max_descent = visual[i].descent;
        }
        width += visual[i].width;
        for (int64_t j = 0; j < visual[i].glyphs.size(); j++) {
            //Calc max offsets for glyphs shifted from baseline and add glyphs width
            if (visual[i].glyphs[j].offset.y > max_pos_offset) {
                max_pos_offset = visual[i].glyphs[j].offset.y;
            }
            if (visual[i].glyphs[j].offset.y < max_neg_offset) {
                max_neg_offset = visual[i].glyphs[j].offset.y;
            }
        }
        visual[i].offset = offset;
        offset += visual[i].width;
    }
    ascent = MAX(max_ascent, -max_neg_offset);
    descent = MAX(max_descent, max_pos_offset);
} else {
    ascent = font->get_ascent();
    descent = font->get_descent();
}
```

gdtl module source code links
general documentation/info links

CC 0